

Ingegneria del Software a.a. 2025-26
Prova Scritta del 22 gennaio 2026 (Gruppo 1) – SOLUZIONE fila A

Quiz di sbarramento

COGNOME

NOME

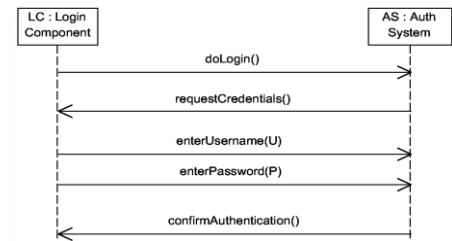
MATRICOLA

Rispondere alle seguenti domande. Per ogni domanda, solo una soluzione è corretta. Il quiz si ritiene superato se si risponde correttamente ad **almeno 7 domande**.

Domanda 1

Il **Sequence diagram UML** a destra modella un possibile scenario di **Login** di una generica applicazione. Volessimo modificarlo per discriminare i casi di successo (autenticazione avvenuta) e di fallimento (autenticazione non avvenuta) che costrutti UML dovremmo utilizzare?

- a) Le activation bars (act)
- b) I message guards (mgua)
- c) **Gli alternative blocks (alt)**
- d) I decision nodes (dec)



Domanda 2

Si consideri il metodo a destra, che restituisce un array contenente i **quadrati** degli elementi dell'array di input e il **test JUnit incompleto** subito sotto. Quale delle seguenti asserzioni **completa correttamente** il test?

- a) assertEquals(new int[]{1, 4, 9}, result);
- b) assertTrue(result == new int[]{1, 4, 9});
- c) assertEquals(input, result);
- d) **assertEquals(new int[]{1, 4, 9}, result);**

```
public static int[] squareAll(int[] a) {
    int[] result = new int[a.length];
    for (int i = 0; i < a.length; i++) {
        result[i] = a[i] * a[i];
    }
    return result;
}

@Test
public void testSquareAll() {
    int[] input = {1, 2, 3};
    int[] result = squareAll(input);

    // COMPLETARE CON L'ASERZIONE CORRETTA
}
```

Domanda 3

Quale è la relazione che intercorre tra un **Caso d'uso** e uno **Scenario**?

- a) Uno Scenario è un insieme di Casi d'uso che hanno in comune lo scopo finale dell'utente
- b) Un Caso d'uso è composto da un solo Scenario
- c) Un Caso d'uso rappresenta un singolo cammino attraverso lo Scenario
- d) **Uno Scenario rappresenta un singolo cammino attraverso il Caso d'uso**

Domanda 4

Nelle applicazioni Web a due livelli '**Thin Client**'

- a) **La capacità elaborativa è principalmente concentrata sul Server**
- b) La capacità elaborativa è principalmente concentrata sul Client
- c) La capacità elaborativa è concentrata in uno strato intermedio separato che gestisce multiutenti
- d) La capacità elaborativa è suddivisa in parti uguali tra Server e Client

Domanda 5

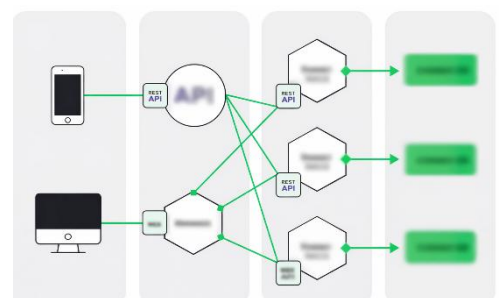
Quale dei seguenti tool introdotti nel Laboratorio 2 del corso di FdIS utilizzeresti per **analizzare l'architettura SW** di un sistema Java e **calcolare alcune metriche strutturali di design** come ad esempio la **coesione**?

- a) SonarQube for IDE
- b) Class Visualizer
- c) **DesigniteJava**
- d) SonarLint

Domanda 6

Quale stile architetturale ha guidato lo sviluppo della seguente applicazione?

- a) Stile P2P (Peer to Peer)
- b) Stile Layered (a livelli)
- c) Stile chiamato Manager model
- d) **Stile basato su Micro-servizi**



Domanda 7

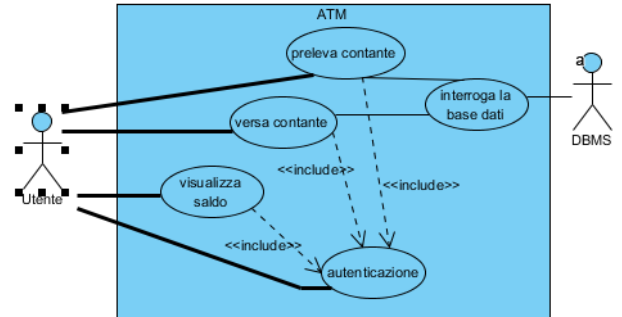
Se abbiamo un programma Java in cui la classe ClassB eredita da ClassA. Quale delle seguenti istruzioni è ERRATA?

- a) ClassA a = new ClassB()
- b) ClassB b = new ClassA()
- c) ClassB a = new ClassB()
- d) ClassA b = new ClassA()

Domanda 8

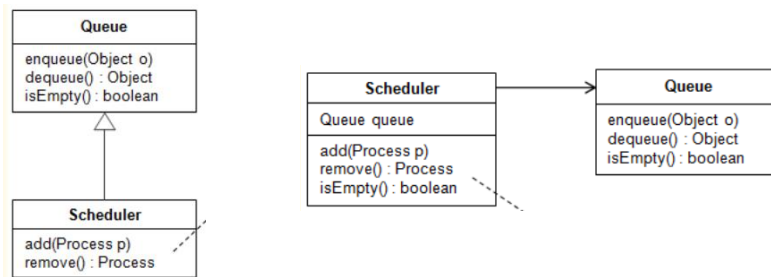
Cosa è rappresentato nella figura mostrata a lato a destra?

- a) Uno use case diagram UML **corretto** di un sistema ATM
- b) Uno use case diagram UML **scorretto** di un sistema ATM. L'errore è nel verso delle frecce delle relazioni 'include' che andrebbe invertita
- c) Uno use case diagram UML **scorretto** di un sistema ATM. L'errore sta nel fatto che "interroga la base dati" non è uno use case
- d) Uno use case diagram UML **scorretto** di un sistema ATM. L'errore sta nel fatto che essendo il DBMS un attore secondario non dovrebbe essere rappresentato con lo stereotipo "stick-man" ma con uno stereotipo apposito per gli attori secondari (ad esempio con un rettangolo)



Domanda 9

Si supponga di avere implementato uno **Scheduler** di processi come una sottoclasse di una **Queue** (vedere figura in basso a sinistra). Lo Scheduler fornisce le operazioni **add()** di un processo e **remove()** di un processo (politica first-in/first-out visto che si tratta di un Queue). Tali operazioni riuseranno le operazioni **enqueue()** e **dequeue()** della **Queue**. Ora si supponga di applicare il refactoring 'Replace Inheritance with Delegation' (vedere figura in basso a destra). Scegliere tra le quattro opzioni quella che corrisponde all'implementazione corretta dell'operazione **remove()** nella classe **Scheduler** dopo che è stato applicato il refactoring



- a) Process **remove()** {return queue.dequeue();}
- b) Process **remove()** {return (Process) queue.dequeue();}
- c) Process **remove()** {return dequeue();}
- d) Process **remove()** {return scheduler.remove();}

Domanda 10

Con riferimento al **Modello di Dominio (Domain Model)** utilizzato nelle prime fasi dell'analisi e del design di un sistema informativo, quale delle seguenti affermazioni **NON** è corretta?

- a) Il Modello di Dominio contribuisce a definire un vocabolario condiviso tra stakeholder e sviluppatori, che può essere riutilizzato nelle fasi successive del design software
- b) Gli elementi rappresentati nel Modello di Dominio corrispondono a concetti rilevanti del dominio applicativo, indipendentemente dalle decisioni di implementazione
- c) Il Modello di Dominio, così come introdotto e discusso a lezione, è tipicamente espresso tramite una rappresentazione visuale basata su classi e relazioni
- d) Il Modello di Dominio fornisce una descrizione strutturata del comportamento e dell'architettura interna del sistema software da realizzare